

PLAN DE REMEDIACIÓN DE VULNERABILIDADES

Proyecto: Control-PrecISO

Documento base: Informe de análisis de vulnerabilidades (security-analysis/INFORME_VULNERABILIDADES.md y versión imprimible asociada)

Versión del plan: 1.0

Fecha: 13 de mayo de 2026

Ámbito: Medidas orientadas a hallazgos **V-001 a V-014**, priorizando controles **implementables en el código** (HTML/JavaScript) del repositorio, sin sustituir la necesidad futura de endurecimiento en **AWS (API Gateway, Lambda, Cognito)** cuando el equipo disponga de acceso a esos entornos.

Nota para el equipo: Este plan está redactado para uso académico y de gestión interna del SGSI. Las tareas deben validarse en ambiente de pruebas antes de desplegar a producción.

1. Propósito y objetivos

1.1 Propósito

Definir un **orden de trabajo, acciones concretas, criterios de verificación** y una **priorización** para reducir los riesgos identificados en el informe de vulnerabilidades, con foco en:

- Hallazgos de **severidad crítica y alta** (orden de atención preferente).
- Controles **factibles desde el código** del front-end actual.
- Trazabilidad entre **ID de hallazgo, archivos y resultado esperado**.

1.2 Objetivos medibles

Objetivo	Indicador sugerido
Reducir superficie XSS	Cero interpolaciones de datos de API en <code>innerHTML</code> sin escape en módulos priorizados (lista en §4).
Endurecer llamadas a API	Todos los <code>fetch</code> hacia <code>execute-api</code> y similares incluyen cabecera <code>Authorization</code> cuando exista sesión.
Eliminar multi-tenant “por defecto”	Ausencia de <code>empresa_id</code> fijo <code>8</code> como fallback en el código versionado.
Integridad de dependencias front	CDN de Bootstrap con SRI (<code>integrity</code> + <code>crossorigin</code>) en todas las plantillas que lo consumen.
Mantenibilidad de autenticación	Una sola implementación de <code>getAccessToken()</code> en <code>js/auth.js</code> .

2. Alcance y exclusiones

2.1 Alcance

- Código bajo `Control-PrecISO-main/` (HTML, JS, CSS).
- Documentación de evidencia en `security-analysis/` .

2.2 Exclusiones (en esta fase del plan)

- Cambios en políticas de **IAM, authorizers** de API Gateway, **WAF** o **Cognito** que requieran consola AWS (se documentan como fase posterior recomendada).
- Pentesting dinámico (DAST) y revisión de infraestructura de red (fuera del alcance del informe estático).

3. Criterios de priorización

Se aplican tres criterios, en este orden:

- 1. **Severidad y riesgo** del informe (crítica → alta → media → baja/informativa).
- 2. **Esfuerzo / riesgo de regresión** (preferencia por cambios localizados y reversibles).
- 3. **Dependencias** (por ejemplo, unificar `authHeaders()` antes de tocar muchos `fetch`).

Leyenda de esfuerzo: B = bajo (horas), M = medio (1–2 días de trabajo focalizado), A = alto (varios días o requiere coordinación con backend).

4. Fases de ejecución (orden recomendado)

Fase 0 — Preparación (B)

ID	Tarea	Responsable sugerido	Verificación
P0-1	Crear rama de trabajo (<code>feature/remediacion-seguridad</code>) y respaldo del estado actual.	Desarrollo	Rama creada; build o prueba manual “smoke” en login y dashboard.
P0-2	Inventariar todos los <code>fetch</code> hacia URLs <code>amazonaws.com</code> y listarlos en una hoja interna.	Desarrollo	Lista completa; marca cuáles ya envían <code>Authorization</code> .
P0-3	Inventariar <code>innerHTML</code> con interpolación <code>\${...}</code> desde datos remotos.	Desarrollo	Lista priorizada por pantallas con datos de usuario o administración.

Fase 1 — Riesgo alto, cambios acotados (B–M)

Orden sugerido dentro de la fase: V-006 → V-005 → V-004 (parcial por archivos críticos) → V-003 → V-014.

Hal laz go	Controles (código)	Archivos / módulos	Esf uer zo	Criterio de “hecho”
V-006	Implementar <code>authHeaders()</code> en <code>auth.js</code> (o <code>utils.js</code>); añadir <code>Authorization: Bearer <access_token></code> a todos los <code>fetch</code> a APIs propias; ante <code>401</code> , redirigir a login o <code>logout()</code> .	<code>superadmin-reportes.html</code> , <code>superadmin-controles.html</code> , <code>superadmin-controles-proveedores.html</code> , y resto de vistas con <code>execute-api</code>	B	Ningún <code>fetch</code> a API de negocio queda sin cabecera cuando hay token; prueba manual con sesión válida e inválida.
V-005	Sustituir <code>innerHTML</code> que incruste <code>error.message</code> por <code>textContent</code> o por <code>escapeHtml(error.message)</code> .	<code>superadmin-reportes.html</code> , <code>superadmin-controles.html</code> , <code>matriz-riesgos.html</code> , etc.	B	Búsqueda en repo sin <code>innerHTML</code> + <code>error.message</code> sin escape.
V-004	Crear <code>js/utils.js</code> con <code>escapeHtml</code> ; incluir script en páginas afectadas; aplicar escape a código, nombre, descripción en plantillas dinámicas; preferir DOM + <code>textContent</code> en nuevas piezas.	<code>gestion-riesgos.html</code> , <code>gestion-proveedores.html</code> , <code>admin-empresas.html</code> , ...	M	Revisión por lista P0-3; prueba con datos de prueba con caracteres <code><></code> en nombres.
V-003	Eliminar <code>localStorage.setItem('empresa_id', 8)</code> y usos <code> 8</code> ; si no hay <code>empresa_id</code> válido, mostrar mensaje y no consumir API con tenant indefinido.	<code>dashboard.html</code> , <code>reportes.html</code> , <code>modificar-proveedor.html</code> , <code>gestion-proveedores.html</code> , etc.	B	No queda el literal de fallback <code>8</code> como tenant por defecto; flujo manual sin id muestra error controlado.
V-014	Unificar una sola función <code>getAccessToken()</code> en <code>js/auth.js</code> ; actualizar llamadas si hiciera falta.	<code>js/auth.js</code>	B	Una definición; pruebas de login y de llamada API.

Fase 2 — Riesgo crítico y medio (mitigación parcial desde front) (M)

Hallazgo	Controles (código)	Archivos / módulos	Esfuerzo	Criterio de “hecho”
V-001	Opción A: no persistir <code>refresh_token</code> en almacenamiento del navegador si el flujo lo permite. Opción B: mover tokens a <code>sessionStorage</code> . Centralizar get/set/clear en <code>auth.js</code> . Quitar logs sensibles.	<code>js/auth.js</code> , referencias en HTML	M	Política documentada en comentario de equipo; sesión probada (login, refresh si aplica, logout).
V-007	Añadir SRI a Bootstrap 5.3.8 en cada <code><link></code> del CDN; <code>crossorigin="anonymous"</code> ; no cambiar versión sin recalcular hash.	Los once HTML reportados por Semgrep (<code>login.html</code> , <code>dashboard.html</code> , ...)	B	Semgrep o búsqueda manual: cada CDN externo tiene <code>integrity</code> .
V-009	Incorporar CSP vía <code><meta http-equiv="Content-Security-Policy" content="..."></code> en plantillas piloto; ajustar <code>script-src</code> , <code>style-src</code> , <code>connect-src</code> para Cognito y API Gateway.	<code>login.html</code> , <code>dashboard.html</code> (piloto) → extensión gradual	M	Sin errores de CSP en consola en flujos login + carga dashboard; documentar política final.
V-002	Tras <code>isAuthenticated()</code> , validar rol con <code>getUserRole()</code> ; si la página exige rol (p. ej. superadmin), redirigir a acceso denegado. Unificar lógica duplicada de rol en <code>dashboard.html</code> .	<code>auth.js</code> , páginas restringidas	M	Usuario sin grupo no accede a rutas de admin en prueba manual.

Fase 3 — Riesgo medio / bajo / informativo (A–B)

Hallazgo	Controles	Esfuerzo	Criterio de “hecho”
V-008	Centralizar URLs y <code>COGNITO_CONFIG</code> en <code>js/config.js</code> ; evitar duplicar strings de endpoint en cada HTML.	B	Un solo lugar de configuración front; revisión de pares.
V-010	Mensajes UX: recordatorio de MFA; limpiar campos de contraseña en error. Cambio de flujo Cognito (SRP, etc.) queda fuera si no hay acceso a consola.	B	Textos acordados con el equipo; sin regresión en registro/login.
V-011	Enlace o instructivo para activación MFA en Cognito (texto en UI).	B	Visible donde defina el curso académico.
V-012	Re-ejecutar Dependency-Check con base NVD actualizada (cuando haya API key o mejor conectividad); registrar fecha en este plan.	A (proceso)	Nuevo <code>dependency-check-report.json</code> archivado.
V-013	Mantener Gitleaks en CI (<code>.gitlab-ci.yml</code> o GitHub Actions cuando migren).	M	Job documentado; último informe sin fugas.

5. Matriz resumen hallazgo → controles → verificación

ID	Severidad	Control principal en código	Verificación rápida
V-001	Crítica	Política de almacenamiento de tokens + centralización	DevTools → Application → sin refresh en localStorage si se adoptó la política; flujo login OK.
V-002	Alta	Chequeo de rol + una sola fuente de verdad en JS	Usuario sin grupo no abre superadmin.
V-003	Alta	Sin fallback <code>tenant</code> 8 ; validación de id	Búsqueda global <code>empresa_id</code> , 8; prueba sin id.
V-004	Alta	<code>escapeHtml</code> / DOM seguro	Payload de prueba con <code><script></code> en nombre no ejecuta.
V-005	Media	Errores sin HTML crudo en DOM	Inspección de plantillas de error.
V-006	Alta	<code>Authorization</code> en <code>fetch</code>	Red sin cabecera no obtiene 200 en APIs que deban estar protegidas (cuando backend exija token).
V-007	Media	SRI en CDN	Atributo <code>integrity</code> presente.

ID	Severidad	Control principal en código	Verificación rápida
V-008	Media	Config centralizada	Un solo módulo de constantes.
V-009	Media	CSP meta (piloto → global)	Consola sin violaciones bloqueantes en flujos críticos.
V-010	Media	UX + higiene de formulario	Revisión de texto y campos.
V-011	Baja	Texto MFA	Visible para el usuario.
V-012	Info	Re-análisis de dependencias	Informe archivado con fecha.
V-013	Info	Gitleaks en CI	Pipeline verde.
V-014	Baja	Una función <code>getAccessToken</code>	<code>auth.js</code> sin duplicado.

6. Riesgos del propio plan de remediación

Riesgo	Mitigación
Regresión funcional tras CSP o SRI	Desplegar primero en piloto; usar lista de verificación de pantallas críticas.
APIs que aún no validan JWT	El front puede enviar token, pero si el backend no valida, el riesgo persiste; registrar como deuda técnica y abrir ítem con equipo de backend/AWS.
Tiempo académico limitado	Cerrar Fase 1 completa antes de ampliar CSP a todas las páginas.

7. Seguimiento y cierre

- Reunión de cierre de fase:** revisar criterios “hecho” y evidencias (capturas, commits, fecha de análisis repetido).
- Actualizar informe:** tras cambios sustanciales, volver a ejecutar Semgrep y Gitleaks y **adjuntar** nuevos reportes en `security-analysis/`.
- Regenerar documentos imprimibles:** ejecutar `python _build_informe_print_html.py` (genera los `.html` para impresión). Para obtener el **PDF del plan**, abra `PLAN_REMEDIACION_print.html` y use **Ctrl+P** → **Guardar como PDF**, o en Windows ejecute `.\generar_pdf_PLAN_REMEDIACION.ps1` en la carpeta `security-analysis/` (requiere Microsoft Edge).

8. Referencias internas

- `security-analysis/INFORME_VULNERABILIDADES.md`
- `security-analysis/semgrep-report.json` / `semgrep-report.txt`
- `security-analysis/gitleaks-report.json`
- `security-analysis/dependency-check-report.html` / `dependency-check-report.json`

Fin del plan de remediación.